

Fiducia: demo video script (3:55 target, 4:00 hard max)

For: Devin, who records it. **Judges:** Superteam Germany, "Build an MVP with Solana at WHU". **Live app:** <https://fiducia-orpin.vercel.app> (Solana **devnet**). **Repo:** <https://github.com/Mulaydm10/solana-hackathon>

Every judging criterion is covered and labelled in the script:

- **[PROBLEM]** a useful idea that solves a clear problem;
 - **[PROTOTYPE]** a working prototype that shows the core idea;
 - **[SOLANA]** a clear role for Solana;
 - **[GROWTH]** who would use it and why, and how we reach the first users.
-

0. Recording setup (do this BEFORE pressing record)

1. Screen:

- 1920×1080, browser zoom 110%, light mode, one clean Chrome window.
- Hide bookmarks, close other tabs. Turn off notifications.

2. Phantom:

- Both test wallets imported (buyer and seller, from `DEVIN-complete-frontend-test.md` Appendix A), on **Solana Devnet**.
- Start on the **buyer** wallet.

3. Tabs, opened in this order:

1. <https://fiducia-orpin.vercel.app/>
2. <https://explorer.solana.com/address/CfD43mq2P1mVVpKxueo1XDe6UrQBCF3DZjNmDGQNVGSV?cluster=devnet> (program)
3. After the prep in step 4, also open these in Explorer tabs: the **release transaction** (from the deal page), the **Deal account**, the **Listing account**, and one **Mandate/agent account** from the mission page

4. Pre-run the slow parts once so the video never waits. Keep the resulting pages ready in tabs:

- **(a) Seller listing:** with the **seller** wallet, sell the ready file `fiducia-demo-weather.csv` (in `~/Downloads`; 310 rows, 6 columns; it grades **A**, tested on the live site) on `/sell`, name "German city weather, Jul–Aug 2026", category `weather`. Note its listing URL.
- **(b) A delivered deal:** with the **buyer**, set up the budget and buy that listing. Switch to **seller**, then Accept and Deliver on the deal page. Switch back to **buyer**. **Don't** get the key yet: you'll do key pickup and release live. Keep that `/deal/<address>` tab.
- **(c) A team mission in progress:** with the **buyer**, hire the Trip planner team (goal below). Sign. Approve Stage 1. Wait until Stage 2 is **waiting for approval**. Keep that `/missions?m=...` tab.

5. Voice: a calm, clear voiceover (or text-to-speech). Each narration block is sized for its timing. Add the **on-screen caption** lines as lower-third text in the edit.

6. Edit: cut any wait longer than 2 s (Phantom popups, RPC). Speed up the signing popups 2x at most. Keep each Phantom approval visible for about 1 s. **Proof matters.**

1. Hook + problem (0:00 – 0:25) [PROBLEM]

Screen: the landing page `/`, slowly scrolling the catalogue.

Narration:

"AI agents can already find data, call APIs and plan whole projects. But the moment they have to pay for something, we stop trusting them. Who checks the data is real? What if the agent overspends, or the seller never delivers? Today the answer is: a human babysits every payment. That doesn't scale."

Caption: Problem: AI agents can't be trusted with money, and buyers can't trust what they buy.

2. What Fiducia is (0:25 – 0:45) [PROBLEM → solution]

Screen: stay on `/`. Hover the three kind badges **Data**, **Service** and **Team**, then the grade badges.

Narration:

"Fiducia is a marketplace where people and their AI agents buy three things: data, services, and entire agent teams. Fiducia, Latin for trust placed in someone who holds your assets. Solana holds the money and enforces every rule. No one has to trust us."

Caption: Fiducia: buy data, services and whole AI agent teams, with every rule enforced on Solana.

3. Trust you can verify: attested listings (0:45 – 1:05) [SOLANA][PROTOTYPE]

Screen:

1. Click "**EU day-ahead power prices**" (grade A).
2. Show the report, the price reasons and "no score yet" reputation.
3. Click a **proof link**, so the explorer opens the listing account. Point at it.
4. Come back.

Narration:

"Every grade here is attested on chain by a registered assessor. The site only shows it if the report's hash matches what's on Solana. Reputation can't be faked either: no score until ten real deals from three different buyers, and it flags when one buyer dominates."

Caption: Grades are attested on-chain. Wash-resistant reputation.

4. Selling, with the method kept private (1:05 – 1:30) [PROTOTYPE]

Screen:

1. Switch Phantom to **seller**. Go to `/sell`.
2. Upload a tiny CSV.
3. Show the step timeline animating: **classify** → **assess (grade)** → **price (low/mid/high and reasons)** → **draft terms**.
4. Click **Sign**, and show the Phantom popup for `create_listing`. Approve it.
5. Cut to the success state.

(If time is tight, show the pre-made listing from 0.4a instead of signing live.)

Narration:

"Selling is one upload. An agent chain classifies the data, grades its quality, suggests a fair price from comparable listings and drafts the terms. The seller signs the listing in their own wallet. The data goes into encrypted custody, so buyers get the exact bytes that were assessed, and the seller never has to reveal how they made it."

Caption: Upload → graded, priced and drafted by agents → seller signs on Solana. Data stays encrypted.

5. Buying with escrow + sealed data (1:30 – 2:10) [SOLANA][PROTOTYPE]

Screen: Phantom on **buyer**.

- (10 s) On the seller's listing, click **Buy**. Show the **"Set up your budget"** card (budget per day, max price), then cut to the **deal draft**: price, seller, deadline, review window.
- (25 s) Switch to the pre-made **delivered deal** tab (0.4b):
 - show the status stepper: **Funded** → **Delivered**;
 - click **Get the data**: Phantom asks to sign a **message**, not a transaction;
 - the file downloads and **"matches the on-chain content hash ✓, key received"** appears;
 - click **Release**, approve in Phantom, and the status becomes **Released**.
- (5 s) Flash the explorer showing the release transaction.

Narration:

"Buying works like a smart escrow. First, the buyer sets an on-chain budget: a daily limit and a max price. The program refuses anything above it. Payment sits in escrow; the seller even stakes. After delivery, the buyer's wallet signs a one-time request, the key arrives sealed to this browser only, and the browser checks the data against the hash on Solana. Happy? Release. If not, challenge it, and an independent verifier rules. No delivery, no payment."

Caption: On-chain budget → escrow → sealed key → verified against the chain → release or challenge.

6. Hire a whole agent team, with safe autonomy (2:10 – 3:05) [SOLANA][PROTOTYPE]

Screen:

- (15 s) `/hire` : type the goal **"Plan a 3-day trip to Lisbon in May under 900 EUR"**. The **Trip planner team** is matched. Show the **mission terms**: roles with per-agent caps, stages with "you approve", budget, expiry. Show the single Phantom popup (mission + fee deal in one transaction). Approve it, or cut to the pre-made mission.
- (25 s) The pre-made mission tab (0.4c) `/missions?m=...` :
 - the **approvals inbox** shows "waiting for you";
 - Stage 2's plan** is written in plain words;
 - click **Approve** and sign;
 - the event feed fills in: **approved** → **each agent's result** → **delivered: final product + hash**. Point out that the researcher's attempt to buy outside its mandate is **refused**: nothing it isn't allowed to pay can leave the vault.

3. (10 s) Hover an agent card: role, address, **spent / cap**, and the **Revoke** button. Say "one click". While hovering, say the **own-VM** line (narration below).
4. (5 s) Final product shown. Click **Release** on the team fee (or show it enabled).

Narration:

"Here's where it gets interesting: you can hire a whole team of AI agents for any goal: research, trading analysis, trip planning. Each agent gets an on-chain mandate: a spending cap, a per-payment cap, allowed payees and an expiry. Nothing runs until you approve each stage, and your approval is bound to the exact plan you saw. A different plan can't reuse it. Every agent runs in its own isolated VM, so a team can set up its environment exactly the way it wants: its own tools, files and settings, without touching anyone else's. Secrets never enter the VM; agents get scoped capabilities instead. Every agent payment has to pass its on-chain mandate. Here, a purchase outside it is simply refused. One click revokes any agent. And the team only gets paid for the exact final product you were shown, with a hash on Solana. To be clear: the agents' intelligence (Claude doing the research and writing) is the next step and isn't plugged in yet. What you see running is the full safety layer around them, live on Solana: budgets, mandates, stage approvals, revoke, escrow and payout."

Caption: Agent teams: each agent in its own VM (custom environment per team) · on-chain mandates · stage-gated · revocable in one click.

7. Solana integration: show it on chain (3:05 – 3:30) [SOLANA]

This is the scene that proves Solana is the core of the product, not decoration. Everything is shown live in **Solana Explorer (devnet)**.

Screen:

1. (6 s) The explorer tab: program `CfD43mq2P1mVVpKxueo1XDe6UrQBCF3DZjNmDGQNVGsv`, **Executable: yes**, **Upgradeable**, devnet. Scroll its **transaction history**: you can see `create_listing`, `attest_listing`, `create_deal`, `submit_delivery`, `release`, `create_mission`, `add_mandate`, `approve_stage`.
2. (6 s) Open **the release transaction from scene 5** (link on the deal page). Point at:
 - the **instruction name** `release`;
 - the **USDC token transfer** from the escrow vault (a program-owned account) to the seller;
 - the **signer**, which is the buyer's Phantom wallet.
3. (6 s) Open **the Deal account** (link on the deal page): its state lives on chain (status *Released*, price, delivery hash). Then the **Listing account**: content hash, assessor, attested.
4. (5 s) Open **the Mission or Mandate account** from scene 6 (agent link on the mission page): cap, spent, stage, revoked = false.
5. (2 s) Quick cut to `/llms.txt`, the machine-readable catalogue agents read.

Narration:

"And this is all real, on Solana. One program holds everything: budgets, listings, escrow, reputation, missions and agent mandates, as program accounts anyone can inspect. Here's the release we just signed: the program moved USDC out of an escrow vault only it controls, straight to the seller. The deal, the listing and every agent's mandate are on-chain state, not rows in our database. Agents use the same rails through our MCP server, and pay per call in USDC over x402. Sub-cent fees and fast finality are what make per-call payments and per-stage escrow viable at all."

Caption: On Solana: 1 program · budgets, escrow, listings, reputation, missions & mandates as on-chain accounts · USDC vaults owned by the program · x402 pay-per-call

If short on time: keep beats 1, 2 and 4 and cut 3 and 5. Never cut beat 2: the escrow-to-seller USDC transfer is the clearest proof.

8. Who it's for + first users + close (3:30 – 3:55) [GROWTH]

Screen: back to / hero. End on the logo/URL card with the repo link.

Narration:

"Our first users are small data sellers, like energy analysts, researchers and scrapers, who want to monetise datasets without giving away their method, and developers building AI agents who need a safe way to let them spend. We reach them through Solana and AI-agent communities, and by listing Fiducia as an MCP server where agent builders already look for tools. Fiducia: trust, enforced on Solana."

Caption (end card):

Fiducia: trust, enforced on Solana
fiducia-orpin.vercel.app · github.com/Mulaydm10/solana-hackathon
Solana devnet · Superteam Germany × WHU 2026

9. Feature checklist the video must show (tick while editing)

- ☐ Problem and target user stated (sec 1, sec 8)
- ☐ Three listing kinds: Data, Service, Team (sec 2)
- ☐ On-chain attested grade + proof link to the explorer (sec 3)
- ☐ Wash-resistant reputation (sec 3)
- ☐ Sell: agent chain steps + seller signs in Phantom + encrypted custody (sec 4)
- ☐ On-chain buyer budget policy (sec 5)
- ☐ Escrow buy in Phantom (sec 5)
- ☐ Sealed key pickup (message signature) + hash verified (sec 5)
- ☐ Release (and mention challenge/verifier) (sec 5)
- ☐ Hire a team: goal → matched blueprint → mission terms → one signature (sec 6)
- ☐ Stage approval bound to the plan + approvals inbox (sec 6)
- ☐ Mandate enforcement visible: an out-of-mandate purchase is refused (sec 6)
- ☐ Own VM per agent: teams keep their environment as they want; no secrets inside (sec 6)
- ☐ Said clearly: agent intelligence (Claude) is not implemented yet; the safety layer is live (sec 6)
- ☐ Per-agent caps + one-click revoke (sec 6)
- ☐ Final product → release the team fee (sec 6)
- ☐ **Solana integration shown in Explorer:** program, release tx (USDC vault → seller, buyer signed), Deal + Listing accounts, Mission/Mandate account (sec 7)
- ☐ For agents: llms.txt, MCP, x402 pay-per-call (sec 7)
- ☐ Why Solana: fees, finality, one auditable program (sec 7)
- ☐ First users + how to reach them (sec 8)
- ☐ URL + repo on the end card (sec 8)

10. Honest limits (don't claim otherwise on camera)

- The agents' outputs are deterministic stand-ins until Claude is plugged in (roadmap). The final product text is short. Show its **hash and the release**, and don't dwell on the text.
- Challenges are ruled by the verifier service; in this build a challenged deal without a ruling refunds at its deadline. **Mention** challenge; don't wait for a ruling on camera.

12. Complete feature inventory (what Fiducia has, and where to show it)

Live and working (verified end to end on the live site, 4 Oct 2026):

#	Feature	Where it's shown	In the video
1	Three listing kinds in one on-chain registry: Data, Service, Team	/ catalogue	sec 2
2	Search and filters (kind, category, price, grade, reputation, attested only)	/	sec 2
3	Agent seller chain: classify → assess (quality grade A–D, PII and secret scan) → price (comparables) → draft terms	/sell	sec 4
4	Seller signs create_listing in Phantom	/sell	sec 4
5	Encrypted custody: data accepted only if it matches the on-chain content hash; key sealed separately	/sell (server)	sec 4
6	Assessor attests on chain; site shows the grade only if the report hash matches the chain and the assessor is registered	/listing/... + explorer	sec 3
7	Wash-resistant reputation (no score < 10 deals / 3 buyers; dominance flag)	/listing/...	sec 3
8	On-chain buyer budget policy (daily budget, max price, optional seller allowlist)	Buy → "Set up your budget"	sec 5
9	Escrow deal: seller stake, deadlines, review window	/deal/...	sec 5
10	Seller accept + deliver (Data delivery must equal the listed content hash)	/deal/... (seller wallet)	prep 0.4b
11	Sealed key pickup: wallet signs a message → key sealed to a one-time browser key → browser verifies against the chain	/deal/...	sec 5
12	Release pays the seller; challenge with a bond (an independent verifier rules)	/deal/...	sec 5
13	Hire a team: goal → matched blueprint → mission terms → mission + fee deal in one signature	/hire	sec 6
14	Per-agent on-chain mandates: cap, per-payment cap, payees, stages, expiry	/hire, /missions	sec 6
15	Stage approvals bound to the exact plan hash; approvals inbox	/missions	sec 6
16	Out-of-mandate payment refused	/missions event feed	sec 6
17	One-click revoke of an agent (one transaction)	/missions	sec 6
18	Final product hash delivered on the fee deal; release only for that hash	/missions	sec 6
19	Own VM per agent (team feature): every agent runs in its own isolated VM/container, so each team keeps its environment as it wants (tools, files, settings) without affecting others; private folder, no network unless granted; secrets via a capability broker (agents never see credentials); prompt-injection quarantine	backend	sec 6 (narration + caption)
20	Demand board (what buyers want but can't find)	/demand	sec 7
21	Seller dashboard	/dashboard	optional
22	Machine-readable catalogue for agents	/llms.txt, /api/catalogue	sec 7

#	Feature	Where it's shown	In the video
23	MCP server for AI agents (find, buy, sell, status, release, challenge, call_service)	repo <code>mcp/</code>	sec 7 (mention)
24	x402 pay-per-call in USDC, no answer = no charge (verified live on devnet)	repo <code>agents/</code>	sec 7 (mention)
25	Devnet faucet for test tokens (rate-limited)	<code>/api/faucet</code>	optional
26	On-chain proof page	<code>/proof</code>	optional
27	One audited Solana program (Anchor), 92 tests incl. randomized attack searches; deployed binary verified byte-for-byte	explorer + repo	sec 7

Not implemented yet (say so in the video):

- **Agent intelligence:** Claude doing the actual research, writing, classification and pricing narratives. Today the agents are deterministic stand-ins inside the full safety layer.
- **Live challenge rulings:** the verifier service is built; wiring it to the site's fact routes is next. Until then a challenged deal refunds at its deadline.
- **Mainnet:** devnet only.

Roadmap line for the video (sec 8, optional, 5 s): "Next: plug Claude into the agents, wire live verifier rulings, and launch on mainnet with real USDC."

11. Rules for the recording

- Devnet only. Never show a private key on screen (blur the Phantom import screen if it's ever visible).
- If something fails live, cut and use the pre-made tab. Never fake a result.
- Keep the total under **4:00**. If over, shorten section 4 to the pre-made listing and section 2 to 10 s. Keep section 7's Explorer beats 1, 2 and 4.
- Export 1080p MP4 and upload it as an **unlisted YouTube** or Loom link, public to anyone with the link.